ProspectIQ — Development Plan & Technical Blueprint

Document type: Initial Product Architecture & Engineering Plan
Version: 1.0
Status: Proposed baseline for development
Purpose: Establish the architecture, module boundaries, implementation approach, engineering principles, and testing strategy for building ProspectIQ as a production-grade application.

---

1. Project Overview

1.1 What is ProspectIQ?

ProspectIQ is an AI-powered B2B sales intelligence platform that helps a seller determine:

- Which companies are worth pursuing?
- What evidence suggests that a company may need the seller's products or services?
- How closely does a potential customer match the seller's business and ideal customer profile?
- Which employees at the target company are relevant decision-makers or influencers?
- What should the seller communicate to each person, and why?

The platform combines seller-provided business information, external company research, evidence validation, lead qualification, decision-maker discovery, and personalized outreach generation.

1.2 Core product principle

ProspectIQ must be an **evidence-driven sales intelligence system**, not merely an LLM-powered search and email generator.

Every important recommendation should be traceable to supporting evidence, and the system must distinguish verified facts, inferred conclusions, missing information, and conflicting claims.

1.3 Product workflow

The overall product follows this sequence:

1. A user registers and establishes an authenticated workspace.

2. The seller configures their company profile, offerings, and ideal customer profile.
3. The user submits a target company for research.
4. The research orchestrator determines what information is needed and invokes appropriate tools.
5. Retrieved information is normalized, validated, attributed to sources, and stored as evidence.
6. The orchestrator identifies missing or weak information and performs additional targeted research.
7. The system combines the seller profile with the lead's evidence to produce a summarized sales intelligence report.
8. Relevant employees and decision-makers are identified and assessed.
9. Personalized outreach drafts are generated for selected employees.
10. The user reviews, edits, approves, or rejects drafts.
11. The application is tested throughout development and undergoes a comprehensive production-readiness assessment before deployment.

---

2. Agreed Development Structure

ProspectIQ will be developed in seven modules or phases.

Module	Name	Primary responsibility
1	Authentication & User Management	Identity, access control, workspace ownership and user lifecycle
2	Tools & Integrations	Search, website extraction, company data, people discovery, document processing and other external capabilities
3	Lead Research & Orchestration	Research planning, tool execution, evidence management, gap detection, validation and research completion
4	Seller Company Profile Setup	Seller information, product/service details, document extraction and ICP configuration
5	Summarized Report Generation	Seller–lead fit, scoring, business insights, evidence-backed summaries and decision-maker presentation
6	Personalized Draft Generation	Employee-specific outreach drafts, personalization, policy checks and human approval
7	Production Testing & Deployment	System-wide testing, security, performance, reliability, deployment, monitoring and operational readiness

Testing rule: Every module must pass its defined completion tests before it is considered complete. Testing continues throughout development; Module 7 performs final cross-module and production-readiness validation.

2.1 High-level system architecture

```
flowchart TD
    U[User] --> API[API and Application Layer]
    API --> AUTH[1. Authentication and User Management]
    API --> SELLER[4. Seller Company Profile]
    API --> RESEARCH[3. Lead Research and Orchestration]
    API --> REPORT[5. Report Generation]
    API --> DRAFT[6. Draft Generation]

    RESEARCH --> TOOLS[2. Tools and Integrations]
    TOOLS --> EXT[External Search, Websites, Company and People Data]

    SELLER --> DB[(Persistent Database)]
    AUTH --> DB
    RESEARCH --> DB
    REPORT --> DB
    DRAFT --> DB
    TOOLS --> STORAGE[(Document and Artifact Storage)]

    RESEARCH --> EVIDENCE[Evidence and Research State]
    EVIDENCE --> REPORT
    SELLER --> REPORT
    REPORT --> DRAFT

    API --> OBS[Logging, Metrics and Tracing]
    RESEARCH --> OBS
    TOOLS --> OBS
    REPORT --> OBS
    DRAFT --> OBS
```

This diagram represents the intended logical architecture, not a requirement to deploy every component as a separate service. The initial implementation should use a **modular monolith** with clear internal boundaries. Separate services can be introduced later when justified by scale or operational needs.

---

3. Module 1 — Authentication & User Management

3.1 Objective

Provide secure identity management and ensure every user's data, research history, company profiles, reports and outreach drafts are accessible only to authorized users.

Authentication answers *who the user is*. Authorization answers *what that user may access or do*.

3.2 Functional requirements

Account lifecycle

- User registration and login.
- Email verification, where applicable.
- Logout and session/token revocation.
- Password reset and account recovery, if password-based authentication is supported.
- Profile viewing and updating.
- Account deactivation and deletion workflows.
- Clear handling of expired, invalid or revoked credentials.

Workspace and access management

- Workspace creation and ownership.
- Workspace membership and invitations if collaboration is supported.
- Roles such as owner, administrator and member where appropriate.
- Permissions for viewing or modifying seller profiles, research jobs, reports and drafts.
- Tenant isolation so one workspace cannot access another workspace's data.

Roles should be introduced only where they correspond to real product permissions. Avoid creating a complicated permission system before it is needed.

3.3 Implementation approach

1. Define the authentication method and threat model.
2. Define user, workspace, membership and permission models.
3. Implement authentication middleware or dependency injection at the API boundary.
4. Establish a single authorization policy for protected resources.
5. Associate business records with their owning workspace.
6. Validate access at the service or repository boundary, not only in the UI.
7. Add rate limits to sensitive endpoints and appropriate audit events.
8. Test unauthorized access, cross-workspace isolation and account lifecycle operations.

Prefer a well-maintained authentication provider or library over custom cryptography or a homegrown authentication protocol.

If token-based authentication is used, define token expiry, refresh and revocation behavior. Store credentials and secrets securely, and never log passwords, access tokens or API keys.

3.4 Proposed data entities

- `User`
- `Workspace`
- `WorkspaceMembership`
- `Role` or permission definitions
- `Session` or refresh-token records, if required by the chosen authentication mechanism
- `AuditEvent`

Authentication-provider-specific fields should remain isolated from the rest of the application.

3.5 Module workflow

```
sequenceDiagram
    actor User
    participant API
    participant Auth as Authentication Service
    participant DB as User Repository

    User->>API: Register or log in
    API->>Auth: Validate credentials or provider token
    Auth->>DB: Retrieve or create user
    DB-->>Auth: User identity and status
    Auth-->>API: Authenticated identity
    API-->>User: Session or access token

    User->>API: Request protected resource
    API->>Auth: Authenticate and resolve workspace
    Auth-->>API: User and permitted scope
    API->>API: Authorize requested operation
    API-->>User: Authorized result or access denied
```

3.6 Design principles and patterns

- **Dependency Inversion:** Application services depend on identity interfaces rather than a specific provider's SDK.
- **Single Responsibility:** Authentication, authorization and business logic remain separate.
- **Adapter Pattern:** Provider-specific authentication is wrapped behind an application-facing interface.

- **Policy/Specification Pattern:** Resource access is evaluated against explicit permissions and ownership rules.

3.7 Completion tests

- Registration, login, logout and recovery flows.
- Invalid credentials, expired sessions and revoked tokens.
- Unauthenticated requests to protected endpoints.
- User A attempting to access User B's or another workspace's records.
- Role and permission enforcement.
- Input validation, rate limiting and audit-event behavior.
- Automated API and integration tests for all supported authentication flows.

Module exit criterion: No protected business operation is accessible without valid authentication and authorization, and tenant isolation is demonstrated by automated tests.

---

4. Module 2 — Tools & Integrations

4.1 Objective

Provide a standardized, replaceable and testable collection of tools that can retrieve information from external sources and perform defined operations for ProspectIQ.

The tools layer should not decide the entire research strategy. It should provide reliable capabilities that the research orchestrator can select and combine.

4.2 Tool categories

A. Company discovery and web search

- General web search.
- Company website discovery.
- Search for company announcements, funding, expansion and hiring.
- Retrieval of relevant news and public documents.

B. Website and document extraction

- Website page retrieval and text extraction.
- Sitemap or selected-page discovery where appropriate.
- PDF and other supported document extraction.
- Normalization of extracted content.
- Metadata capture, including URL, retrieval time and content type.

C. Company data

- Company identity and domain resolution.
- Company size, industry, headquarters and business description.
- Funding, growth or other available firmographic data.
- Technology signals where reliable providers are available.

D. People and professional information

- Employee and decision-maker discovery.
- Job title and department retrieval.
- Public professional profile references.
- Source and freshness information for people records.

People data must be obtained from authorized sources and handled according to applicable privacy requirements and provider terms.

E. LLM and model interfaces

- Structured extraction from retrieved content.
- Classification and relevance evaluation.
- Research planning assistance, where appropriate.
- Summarization and evidence interpretation.

The LLM is not itself a source of verified company facts. Any factual conclusion must be grounded in retrieved evidence or explicitly identified as an inference.

4.3 Standard tool contract

Every tool should follow a consistent interface, regardless of its provider.

Illustrative Python contract:

```
from typing import Protocol, Any
from pydantic import BaseModel


class ToolResult(BaseModel):
    success: bool
    data: dict[str, Any] | list[Any] | str | None = None
    source: str | None = None
    error_code: str | None = None
    retryable: bool = False


class ResearchTool(Protocol):
    name: str
```

```
    async def execute(
        self,
        request: dict[str, Any],
    ) -> ToolResult:
        ...
```

This is a starting contract, not a final universal schema. Individual tools should use validated request and response models rather than leaving all payloads as unrestricted dictionaries.

A tool should define:

- A stable tool name and version.
- Validated input and output schemas.
- Required credentials and configuration.
- Timeout and retry policy.
- Rate-limit behavior.
- Error categories.
- Source attribution and retrieval metadata.
- Logging and observability requirements.
- Data usage restrictions.

4.4 Tool execution architecture

```
flowchart TD
    O[Research Orchestrator] --> R[Tool Registry]
    R --> S[Select Tool Adapter]
    S --> V[Validate Input Schema]
    V --> P[Apply Policy and Execution Limits]
    P --> A[Provider Adapter]
    A --> EXT[External Provider or Source]
    EXT --> N[Normalize Response]
    N --> Q[Validate Output Schema]
    Q --> T[Attach Provenance and Retrieval Metadata]
    T --> RES[Structured Tool Result]
    RES --> O

    A -. transient failure .-> ERR[Classify Error]
    ERR --> RETRY[Bounded Retry if Safe]
    RETRY --> A
    ERR --> FAIL[Return Typed Failure]
    FAIL --> O
```

4.5 Implementation rules

- Implement provider-specific code behind adapters.
- Keep tool selection and research policy outside individual provider adapters.
- Use dependency injection to supply tool implementations.

- Register tools through an explicit registry rather than scattering provider-specific conditionals throughout the application.
- Use bounded timeouts and retries with exponential backoff and jitter where appropriate.
- Apply rate limits and concurrency limits per provider.
- Cache only data for which caching is appropriate and permitted.
- Use circuit breakers or temporary provider suppression when repeated failures justify them.
- Validate response schemas before data reaches the research state.
- Record the source, retrieval timestamp and relevant provider metadata.
- Make mock implementations available for deterministic testing.

Security requirements

Website and URL-fetching tools must defend against server-side request forgery (SSRF), unsafe redirects, access to internal network addresses, DNS rebinding risks and unexpectedly large responses. Apply URL validation, DNS/IP checks, redirect revalidation, response-size limits and safe content-type handling.

External documents and web pages must be treated as untrusted input. Instructions embedded in retrieved content must never override the application's system policies or authorization rules.

4.6 Design principles and patterns

- **Adapter Pattern:** Isolate each external provider.
- **Strategy Pattern:** Allow alternative implementations for a capability.
- **Registry/Factory Pattern:** Resolve a named capability to an approved implementation.
- **Open/Closed Principle:** Add new providers without rewriting the orchestrator.
- **Interface Segregation:** Keep contracts specific to each capability; do not force every tool to implement unrelated methods.

4.7 Completion tests

- Unit tests for input validation, output normalization and error classification.
- Contract tests for every tool implementation.
- Integration tests against provider sandbox environments where available.
- Mock-based tests for rate limits, timeouts, invalid responses and outages.
- Retry and concurrency-limit tests.
- Security tests for malicious URLs, redirects and oversized responses.
- Verification that provenance survives normalization.
- Tests ensuring secrets never appear in logs.

Module exit criterion: Every registered tool has a defined contract, predictable failures, appropriate security controls and automated tests independent of the live provider.

---

5. Module 3 — Lead Research & Orchestration

5.1 Objective

This is the central intelligence workflow of ProspectIQ.

The module determines what is known about a target company, what remains unknown, which sources or tools are appropriate, how retrieved information should be validated, and when research is sufficiently complete to produce a report.

It must support iterative, gap-driven research rather than a fixed sequence of searches.

5.2 Core responsibilities

1. Accept a research request and resolve the target company.
2. Establish the required research fields and their definitions.
3. Create a structured research state.
4. Assess which fields are missing, weakly supported, stale or contradictory.
5. Generate research tasks for those gaps.
6. Select appropriate tools for each task.
7. Execute research within bounded limits.
8. Convert raw results into structured evidence.
9. Validate evidence and link it to the claims it supports.
10. Update field-level research status.
11. Repeat targeted retrieval when useful.
12. Stop when completion criteria or resource limits are reached.
13. Persist the research state, evidence and final status.

5.3 Define the research schema first

Before the orchestrator can reason about completeness, the application must define what information it expects to collect.

Example research fields:

Field	Example	Validation considerations
Company identity	Legal or trading name, website	Identity and domain consistency

Industry	Industry classification	Supported by reliable evidence
Company size	Employee range	Source, freshness and confidence
Business model	Products, services and customer segments	Evidence supports description
Growth signals	Funding, expansion, revenue announcements	Date and source credibility
Hiring signals	Open roles or hiring activity	Current and attributable evidence
Technology signals	Technologies used	Direct evidence versus inference
Strategic initiatives	Announced priorities or investments	Evidence and publication date
Business challenges	Possible operational or commercial needs	Separate facts from hypotheses
Relevant employees	Names, roles and professional references	Identity confidence and source

The exact fields and requirements should be configurable by research purpose. Some fields may be optional or inapplicable to certain companies.

Each field should define:

- A stable field identifier.
- Its meaning and accepted data types.
- Whether it is required.
- Its validation rules.
- Freshness requirements, if applicable.
- Minimum acceptable evidence requirements.
- Whether conflicting evidence requires resolution.

5.4 Evidence is a first-class entity

A research claim must not exist as an unsupported string extracted from an LLM response.

A proposed evidence record should contain:

Evidence
├── evidence_id
├── workspace_id

├── research_id
├── target_company_id
├── field_key
├── extracted_value
├── normalized_value
├── source_url_or_reference
├── source_type
├── source_published_at
├── retrieved_at
├── supporting_excerpt
├── extraction_method
├── confidence
├── validation_status
└── provenance_metadata

The source publication date and retrieval date must remain distinct. A page retrieved today may contain information published years ago.

Evidence confidence is not equivalent to truth. Confidence scores should have defined meanings and should not imply statistical calibration unless calibration has been evaluated.

5.5 Research state

The orchestrator should maintain explicit, serializable state rather than relying on the conversational history of an LLM.

A proposed `ResearchState` includes:

- Research ID and target company identity.
- Requested research objective.
- Research schema and required fields.
- Current field values.
- Evidence records and claim-to-evidence links.
- Field-level statuses.
- Detected gaps and contradictions.
- Pending and completed research tasks.
- Tool execution history.
- Attempt counts and resource budgets.
- Current phase and terminal status.
- Warnings, errors and timestamps.

Suggested field statuses:

- `missing`
- `retrieved`
- `validated`
- `needs_more_evidence`

- `conflicting`
- `not_found`
- `not_applicable`

Suggested research statuses:

- `queued`
- `running`
- `awaiting_retry`
- `completed`
- `completed_with_gaps`
- `failed`
- `cancelled`

Terminal states must be explicit. A company should not be marked fully researched simply because a search provider returned an empty result.

5.6 End-to-end orchestration workflow

```
flowchart TD
    START[Research Request] --> RESOLVE[Resolve Target Company]
    RESOLVE --> INIT[Load Research Schema and Initialize State]
    INIT --> GAP[Analyze Required Fields and Evidence]

    GAP --> DECIDE{Are there actionable gaps?}
    DECIDE -- No --> READY[Check Completion Criteria]
    DECIDE -- Yes --> PLAN[Create Targeted Research Tasks]

    PLAN --> SELECT[Select Approved Tools]
    SELECT --> EXEC[Execute Tools Within Budgets]
    EXEC --> RAW[Collect Raw Results and Source Metadata]
    RAW --> EXTRACT[Extract Structured Claims]
    EXTRACT --> VALIDATE[Validate Schema, Source and Claim Support]

    VALIDATE --> UPDATE[Update Evidence and Research State]
    UPDATE --> GAP

    READY --> COMPLETE{Completion threshold met?}
    COMPLETE -- Yes --> DONE[Persist Completed Research]
    COMPLETE -- No, budget remains --> PLAN
    COMPLETE -- No, budget exhausted --> PARTIAL[Persist Incomplete Research with Gaps]

    EXEC -. provider failure .-> ERROR[Classify and Record Failure]
    ERROR --> RETRY{Retry safe and budget remains?}
    RETRY -- Yes --> EXEC
    RETRY -- No --> UPDATE
```

Important workflow behavior

The loop is not intended to run indefinitely. Each iteration must either produce useful new evidence, resolve a gap, change a field's status, or terminate because further research is not justified.

When a provider fails, the system must distinguish a provider error from a genuine absence of information.

5.7 Detailed implementation stages

Stage A — Resolve and initialize

- Normalize the company name and website.
- Resolve the likely target entity.
- Detect ambiguity between similarly named companies.
- Request clarification if the target cannot be resolved safely.
- Load the required research schema.
- Create the research record and initial state.

Stage B — Assess evidence gaps

For each required field, determine:

- Is a value available?
- Is it supported by evidence?
- Is the evidence credible and sufficiently recent?
- Are there conflicting values?
- Is the field mandatory?
- Would further research plausibly improve the result?

Produce a structured list of research gaps rather than a free-form prompt.

Stage C — Plan research tasks

A research task should identify:

- Target company and field.
- The question to answer.
- The desired evidence type.
- Candidate tools or source types.
- Priority.
- Success criteria.
- Execution budget.

For example, if the employee count is missing, search for company firmographics or a credible company profile rather than repeating general company-description searches.

The LLM may propose tasks, but application code must validate them against the schema, permitted tool registry, security policies and execution budgets.

Stage D — Select and execute tools

The orchestrator selects suitable tools using explicit capability metadata.

A search tool may locate relevant pages. A website reader may retrieve and extract content. A company-data provider may return structured firmographics. A people-data provider may identify relevant employees.

Tools should return standardized results. The orchestrator should not need to understand every provider's proprietary response format.

Stage E — Extract claims and attach evidence

- Preserve the original source reference.
- Extract the relevant passage or structured record.
- Normalize the value into the expected field type.
- Associate the claim with one or more evidence records.
- Record retrieval and publication dates where available.
- Reject malformed or unsupported structured output.

LLM extraction must not invent a source, quote or publication date. If source-level support cannot be established, the claim must remain unverified.

Stage F — Validate evidence

Validation should include separate checks:

1. **Schema validation:** Does the value match the expected type and format?
2. **Source validation:** Is the source retrievable and relevant to the claim?
3. **Semantic validation:** Does the source content actually support the extracted claim?
4. **Identity validation:** Does the evidence refer to the correct company?
5. **Freshness validation:** Is the evidence sufficiently recent for this field?
6. **Consistency validation:** Does it contradict other evidence?
7. **Completeness validation:** Does the field meet its required evidence standard?

A valid JSON response is not proof that the underlying factual claim is correct.

Stage G — Update state and decide what comes next

After validation:

- Accept evidence-backed values.
- Keep weak evidence with an explicit warning or lower status.
- Mark contradictions for resolution.
- Identify fields that still require research.
- Avoid repeatedly querying the same sources without a reason.

- Record unsuccessful attempts and why they failed.
- Recalculate research priorities.

The system should preserve competing claims when appropriate rather than silently replacing one with another.

Stage H — Stop and finalize

Research can stop when:

- All mandatory fields meet their completion criteria; or
- Remaining gaps have been reasonably exhausted; or
- Time, cost, iteration or provider limits are reached; or
- The user cancels the operation.

The final result must distinguish `completed` from `completed_with_gaps`. Any unresolved uncertainty should be available to the report-generation module.

5.8 Validation and completion model

Use field-level completion rather than a single simplistic overall percentage.

For example:

Field: employee_count
Status: validated
Value: 250–500
Evidence: 2 supporting sources
Freshness: acceptable
Conflicts: none

Field: technology_stack
Status: needs_more_evidence
Value: possible use of Technology X
Evidence: 1 indirect reference
Freshness: acceptable
Conflicts: none
Next action: seek direct confirmation or retain as an unverified signal

If an overall completeness score is needed, define its formula explicitly and keep it separate from evidence credibility and sales-fit scoring.

5.9 Reliability and execution controls

- Limit total research duration, iterations, provider calls and estimated cost.
- Apply timeouts to individual tool calls.
- Retry only failures that are transient and safe to retry.

- Use idempotency keys for operations that may be submitted more than once.
- Persist state at meaningful checkpoints.
- Prevent duplicate concurrent research jobs for the same request unless intentionally allowed.
- Support cancellation and recoverable failures.
- Make completed research reusable where permitted and sufficiently fresh.
- Record tool inputs, outputs and errors with appropriate redaction.
- Never let retrieved content directly execute arbitrary code or unauthorized tools.

5.10 Design principles and patterns

- **Strategy Pattern:** Select between alternative research tools.
- **Adapter Pattern:** Normalize provider-specific behavior.
- **State Pattern:** Represent explicit research lifecycle transitions.
- **Pipeline Pattern:** Separate extraction, normalization, validation and persistence.
- **Repository Pattern:** Isolate state and evidence persistence.
- **Specification/Policy Pattern:** Define field completion and evidence acceptance rules.
- **Dependency Inversion:** The orchestrator depends on tool and repository interfaces.
- **Single Responsibility:** Planning, execution, evidence validation and scoring remain distinct responsibilities.

A framework such as LangGraph can be introduced if durable checkpoints, branching, recovery or complex workflow state justify it. The architecture must not depend on a framework to define the business rules.

5.11 Completion tests

- Research state initialization and valid lifecycle transitions.
- Correct gap detection for missing, stale and contradictory fields.
- Tool selection based on task requirements.
- Structured extraction with malformed or incomplete responses.
- Source attribution and claim-to-evidence traceability.
- Detection of unsupported claims and company-identity mismatches.
- Retry behavior, timeout handling and provider failures.
- Maximum iteration and cost-budget enforcement.
- Partial completion and explicit unresolved gaps.
- Persistence and recovery after interrupted execution.
- Duplicate-job and concurrent-update behavior.
- Prompt injection and untrusted-source handling.

Module exit criterion: Research can be executed deterministically within defined limits, evidence is traceable, gaps drive additional retrieval, and the system accurately reports both completed and unresolved fields.

---

6. Module 4 — Seller Company Profile Setup

6.1 Objective

Build a structured understanding of the seller's business so ProspectIQ can determine which leads are relevant to that particular seller.

A general company research result is not sufficient. The platform needs to understand what the seller offers, whom they serve and which problems they solve.

6.2 Functional requirements

Company profile

- Company name, website, industry and description.
- Business model and geographic coverage.
- Company size or operating context, where relevant.

Products and services

- Product or service names.
- Features and capabilities.
- Target customer problems.
- Business outcomes and differentiators.
- Limitations or unsuitable use cases.
- Pricing or commercial constraints, if supplied.

Ideal customer profile

- Target industries.
- Company size and geography.
- Relevant departments and job roles.
- Business problems and buying signals.
- Exclusion criteria.
- Preferred company characteristics.

Document ingestion

- Upload supported files.
- Validate file type, size and permissions.
- Extract text from supported documents.
- Preserve page, section or other source references when available.
- Extract structured seller facts.

- Allow users to review, correct and approve extracted information.
- Track source documents and profile versions.

6.3 Document-processing workflow

```
flowchart TD
    UPLOAD[Upload Seller Document] --> SEC[Validate File, Type and Access]
    SEC --> STORE[Store Original Securely]
    STORE --> EXTRACT[Extract Text and Document Structure]
    EXTRACT --> PARSE[Extract Candidate Company, Product and ICP Facts]
    PARSE --> NORMALIZE[Normalize into Typed Schemas]
    NORMALIZE --> VALIDATE[Validate Required Fields and Conflicts]
    VALIDATE --> REVIEW[User Reviews Extracted Information]
    REVIEW --> EDIT[Correct or Reject Candidate Facts]
    EDIT --> SAVE[Save Approved Profile Version]
    SAVE --> READY[Available to Research and Reporting]
```

6.4 Implementation approach

- Keep file storage separate from relational profile data.
- Use an abstraction for document extraction so parsers can be replaced.
- Store extracted candidate facts separately from approved facts.
- Record source document and location references for extracted claims.
- Validate document processing outputs using schemas.
- Treat uploaded files and their contents as untrusted.
- Avoid treating instructions inside uploaded documents as application instructions.
- Keep a version history of approved seller profiles.
- Re-run affected downstream analysis when material profile information changes.

A user must be able to correct inaccurate extraction. The LLM must not silently overwrite approved company information.

6.5 Proposed data entities

- `SellerCompany`
- `SellerProduct`
- `SellerICP`
- `SellerProfileVersion`
- `SourceDocument`
- `ExtractedFact`
- `ApprovedProfileFact`

6.6 Design principles and patterns

- **Strategy Pattern:** Support multiple document parsers.

- **Adapter Pattern:** Isolate file extraction libraries and external processing services.
- **Repository Pattern:** Separate profile persistence from profile-processing logic.
- **Single Responsibility:** Uploading, extracting, validating and approving facts are separate operations.
- **Human-in-the-Loop:** The seller controls the approved representation of their business.

6.7 Completion tests

- Supported file types, unsupported files and oversized files.
- Extraction from text-based documents and scanned documents where OCR is supported.
- Malformed documents and extraction-provider failures.
- Schema validation and conflicting extracted facts.
- Source references for extracted information.
- Approval, correction, versioning and access control.
- Cross-workspace document isolation.
- Secure file access and retention/deletion behavior.

Module exit criterion: Seller information can be captured, extracted, reviewed, approved and versioned, and the resulting structured profile is safely available to downstream modules.

---

7. Module 5 — Summarized Report Generation

7.1 Objective

Combine the seller's approved company profile with the lead's validated research and evidence to generate an actionable, explainable sales intelligence report.

This module consumes research results. It must not silently substitute for the research orchestrator or fabricate missing facts.

7.2 Report contents

Company overview

- Company identity, industry, size and business model.
- Relevant recent announcements and strategic initiatives.
- Important hiring, expansion or technology signals where evidence exists.

Evidence-backed observations

- Observed business signals.
- Possible needs or challenges.
- Supporting evidence and source references.
- Evidence dates and confidence or validation status.
- Unresolved gaps and contradictory information.

Seller–lead fit

- Alignment between the seller's offerings and the lead's likely needs.
- ICP fit.
- Relevant departments and potential use cases.
- Reasons to pursue or deprioritize the company.
- Missing information that affects the recommendation.

Decision-maker summary

- Relevant employees and job titles.
- Departments and likely responsibilities.
- Public professional references and source information.
- Relevance to the seller's offering.
- Confidence and uncertainty in role identification.

Recommended action

- Priority recommendation.
- Evidence supporting the recommendation.
- Suggested conversation angle.
- Key risks or unanswered questions.
- Recommended next step.

7.3 Separate different kinds of scores

Do not collapse all assessments into one opaque score.

Keep separate measures for:

1. **Research completeness:** How much of the required research schema is covered?
2. **Evidence quality:** How well are the important claims supported?
3. **Lead qualification:** How promising is the company based on observable buying or growth signals?
4. **Seller–lead fit:** How closely does the lead match the seller's offerings and ICP?
5. **Contact relevance:** How relevant is a person to the potential buying process?

Each measure needs a documented definition. If a score is not calibrated against historical outcomes, present it as a heuristic indicator rather than a probability of conversion.

7.4 Report-generation workflow

```
flowchart TD
    R[Validated Lead Research] --> CHECK[Validate Research State and Evidence]
    S[Approved Seller Profile] --> CHECK
    P[Approved Scoring Policies] --> CHECK

    CHECK --> FEATURES[Build Structured Report Features]
    FEATURES --> FIT[Evaluate ICP and Offering Fit]
    FEATURES --> QUALIFY[Evaluate Lead Signals]
    FEATURES --> PEOPLE[Assess Available Decision-Makers]

    FIT --> SCORE[Calculate Explainable Scores]
    QUALIFY --> SCORE
    PEOPLE --> ASSEMBLE[Assemble Structured Report]
    SCORE --> ASSEMBLE

    ASSEMBLE --> GEN[Generate Evidence-Constrained Narrative]
    GEN --> VERIFY[Validate Claims, References and Output Schema]
    VERIFY --> PASS{Report valid?}

    PASS -- Yes --> SAVE[Persist Report and Version]
    PASS -- No --> REPAIR[Bounded Repair or Validation Failure]
    REPAIR --> VERIFY

    SAVE --> PRESENT[Present Report to User]
```

7.5 Implementation approach

1. Validate that research and seller-profile versions exist.
2. Build a structured report input from approved facts and evidence.
3. Calculate deterministic metrics in application code where possible.
4. Use explicit scoring policies and version them.
5. Ask the LLM to generate narrative sections from the structured input.
6. Validate the report against a schema.
7. Check that factual statements and cited references are grounded in the supplied evidence.
8. Mark unsupported conclusions or omit them.
9. Persist the report with links to the profile, research and scoring-policy versions.
10. Allow the report to be regenerated when relevant inputs change.

The LLM should explain the findings; it should not be the sole authority for numerical scores or eligibility rules.

7.6 Proposed data entities

- `LeadReport`

- `ReportSection`
- `LeadScore`
- `ScoringPolicyVersion`
- `ReportEvidenceReference`
- `DecisionMaker`
- `ReportVersion`

7.7 Design principles and patterns

- **Strategy Pattern:** Replace or configure scoring policies.
- **Pipeline Pattern:** Separate feature building, scoring, narrative generation and validation.
- **Template Method or Composition:** Maintain consistent report structure while allowing optional sections.
- **Repository Pattern:** Store reports and their lineage.
- **Single Responsibility:** Scoring logic must not be mixed with LLM prose generation.

7.8 Completion tests

- Reports generated from complete and incomplete research.
- Seller–lead fit under different ICP configurations.
- Deterministic score calculations and boundary conditions.
- Correct propagation of evidence references.
- Detection of unsupported factual claims.
- Contradictory or stale evidence handling.
- Structured-output validation and bounded repair.
- Report versioning and regeneration.
- Authorization and tenant isolation.
- Reproducibility using the same inputs and policy version where deterministic behavior is expected.

Module exit criterion: The report explains why a lead may or may not be valuable to a particular seller, with traceable evidence, transparent scoring and explicit uncertainty.

---

8. Module 6 — Personalized Draft Generation

8.1 Objective

Generate distinct, relevant outreach drafts for appropriate employees at a researched lead company.

Each draft should reflect the recipient's professional role, the lead's evidence-backed context and the seller's approved offering. The application must preserve human control over external communication.

8.2 Inputs

- Approved seller company and product profile.
- Approved ICP and outreach policies.
- Validated lead research and report.
- Relevant decision-maker records.
- Evidence-backed personalization signals.
- Selected outreach channel and desired purpose.
- Existing contact or suppression preferences, where applicable.

8.3 Draft-generation workflow

```
flowchart TD
    REPORT[Approved Lead Report] --> CONTACTS[Select Relevant Employees]
    SELLER[Approved Seller Offering] --> CONTEXT[Build Outreach Context]
    CONTACTS --> CONTEXT
    EVIDENCE[Verified Lead Evidence] --> CONTEXT

    CONTEXT --> ANGLE[Determine Recipient-Specific Value Proposition]
    ANGLE --> PROMPT[Generate Structured Draft]
    PROMPT --> VALIDATE[Validate Claims, Tone and Format]
    VALIDATE --> POLICY[Check Consent, Suppression and Outreach Policies]

    POLICY --> DECIDE{Draft eligible for review?}
    DECIDE -- Yes --> SAVE[Save Draft Version]
    DECIDE -- No --> BLOCK[Block or Flag for Correction]

    SAVE --> REVIEW[Human Review]
    REVIEW --> EDIT[Edit, Approve or Reject]
    EDIT --> AUDIT[Record Decision and Version]
    AUDIT --> READY[Approved Draft Available for Authorized Use]
```

8.4 Implementation approach

A. Select recipients

- Identify people whose professional roles align with the offering.
- Distinguish decision-makers, influencers and other relevant stakeholders.
- Avoid treating every employee as a suitable recipient.

- Preserve source references and identity confidence.
- Avoid inventing email addresses or professional histories.

B. Build a recipient-specific context

For each person, construct a bounded context containing:

- Verified identity and professional role.
- Relevant company initiatives or business signals.
- The seller offering most relevant to that role.
- Supported personalization facts.
- Approved messaging and brand constraints.
- Explicitly excluded claims or unsupported assumptions.

C. Generate the draft

- Choose an appropriate message type and channel.
- Generate a clear subject line when relevant.
- Use a concise, role-specific value proposition.
- Ground personalization in available evidence.
- Avoid false familiarity or claims that the sender has a relationship with the recipient.
- Use a structured response schema.

D. Validate the draft

Validate:

- Correct recipient and company.
- Supported factual claims.
- Relevance to the seller's offering.
- Appropriate tone, length and formatting.
- No unsupported promises or fabricated results.
- No sensitive or inappropriate inferences.
- Applicable consent, suppression, opt-out and provider requirements.

E. Human review

The user should be able to:

- Review the draft and its supporting personalization facts.
- Edit or regenerate it.
- Approve or reject it.
- View version history.
- See any policy warnings.
- Export or send it only through an explicitly authorized integration.

Draft generation and actual sending must be separate operations. Approval should not be assumed from generation, and approval should not bypass applicable suppression or consent checks.

8.5 Proposed data entities

- `OutreachDraft`
- `DraftVersion`
- `DraftRecipient`
- `PersonalizationFact`
- `OutreachPolicyResult`
- `ApprovalDecision`
- `SuppressionRecord`, where applicable
- `DeliveryAttempt`, if sending is later implemented

8.6 Design principles and patterns

- **Strategy Pattern:** Support different channels, message types or writing styles.
- **Template/Composition Pattern:** Combine shared brand rules with recipient-specific context.
- **Policy/Specification Pattern:** Enforce outreach eligibility rules.
- **Human-in-the-Loop:** Require explicit approval before sending.
- **Single Responsibility:** Recipient selection, personalization, generation, validation and sending remain separate.

8.7 Completion tests

- Drafts for different roles at the same company are meaningfully differentiated.
- Personalization facts map to supporting evidence.
- Missing or uncertain information does not become fabricated content.
- Structured output, tone and length constraints.
- Policy blocks and suppression handling.
- Draft edits, regeneration, approval and rejection.
- Version history and audit events.
- Authorization and tenant isolation.
- Sending cannot occur through draft generation alone.
- Integration tests for any explicitly supported sending provider.

Module exit criterion: Each draft is traceable to its inputs, validated against applicable policies, stored with its version history and subject to explicit human approval.

---

9. Module 7 — Production Testing & Deployment

9.1 Objective

Validate the entire application and deploy it safely to the target VPS with operational monitoring, recovery procedures and documented release practices.

This module is a dedicated final release phase, but testing begins within every earlier module.

9.2 Testing strategy

Tests performed during each module

Every module must include:

- Unit tests for business logic and validation.
- Integration tests for its dependencies and persistence.
- API contract tests for exposed endpoints.
- Authorization and tenant-isolation tests where relevant.
- Error-path and negative tests.
- Regression tests for previously fixed defects.
- Documentation updates for behavior and known limitations.

Final system-wide tests

Test category	Purpose
End-to-end	Validate complete user journeys across all modules
API integration	Verify request/response contracts, authentication and error behavior
Database integrity	Validate constraints, migrations, transactions and concurrent updates
Security	Test authorization, tenant isolation, injection risks, SSRF, secrets handling and dependencies
Performance	Measure latency, throughput, resource consumption and bottlenecks
Load and stress	Test expected load and behavior beyond expected capacity
Reliability	Test provider outages, timeouts, retries, recovery and partial failures

AI quality	Evaluate extraction accuracy, evidence grounding, gap resolution, scoring consistency and draft quality
Regression	Ensure new changes do not break existing functionality
Operational readiness	Verify logs, metrics, alerts, backups and recovery procedures

The exact tools and thresholds should be selected based on the actual stack and deployment requirements.

9.3 End-to-end test scenario

A complete production-style test should cover:

1. Register a user and create a workspace.
2. Configure the seller's profile and upload a supported document.
3. Review and approve the extracted company information.
4. Submit a target company for research.
5. Verify that the orchestrator identifies missing fields and invokes appropriate tools.
6. Confirm that retrieved claims retain source references and validation statuses.
7. Confirm that research stops within configured resource limits.
8. Generate a report combining the approved seller profile and lead evidence.
9. Verify scores, supporting evidence and unresolved gaps.
10. Identify relevant employees.
11. Generate differentiated outreach drafts.
12. Review, edit, approve and reject drafts.
13. Verify that unauthorized users cannot access any of these records.
14. Simulate provider failures and confirm recoverable behavior.
15. Confirm that logs, metrics and audit events capture the relevant operations without exposing secrets.

9.4 API testing

For every endpoint, document and test:

- HTTP method and route.
- Authentication and authorization requirements.
- Request schema and validation rules.
- Response schema and status codes.
- Error responses.
- Pagination and filtering where applicable.
- Idempotency for operations that may be repeated.
- Rate limits and execution timeouts.
- Tenant isolation.
- Expected side effects and persistence behavior.

Automate the API suite in continuous integration where practical.

9.5 Performance and load testing

Measure realistic workflows, not only isolated endpoints.

Important areas include:

- Concurrent research jobs.
- Slow or rate-limited providers.
- Long-running document extraction.
- Database query latency.
- LLM response time and token/cost usage.
- Worker concurrency and queue backlog, if background jobs are used.
- Memory, CPU and storage usage.
- Response latency for user-facing endpoints.

Define measurable service objectives before release, including expected throughput, latency targets, acceptable error rates and resource limits. Do not claim performance readiness without running tests against representative workloads.

9.6 Deployment architecture

A reasonable initial VPS deployment may include:

- Reverse proxy with HTTPS.
- Application API process.
- Background worker process if long-running research is handled asynchronously.
- Relational database, managed externally or hosted with appropriate isolation and backups.
- Object storage for uploaded documents and artifacts.
- Secure secret configuration.
- Centralized application logs and metrics.
- Automated backup and restore procedures.

The exact topology depends on the chosen database, storage and job-processing technologies. Do not introduce extra infrastructure unless it solves a real requirement.

Deployment flow

```
flowchart TD
    DEV[Development Branch] --> CI[Automated CI Checks]
    CI --> UNIT[Unit, Integration and Contract Tests]
    UNIT --> SEC[Security and Dependency Checks]
    SEC --> BUILD[Build Versioned Artifact]
    BUILD --> STAGE[Deploy to Staging]
    STAGE --> SMOKE[Smoke and End-to-End Tests]
    SMOKE --> GATE{Release Approved?}
```

```
GATE -- No --> FIX[Fix and Re-test]
FIX --> CI

GATE -- Yes --> BACKUP[Verify Backup and Rollback Readiness]
BACKUP --> PROD[Deploy to Production VPS]
PROD --> HEALTH[Health Checks and Smoke Tests]
HEALTH --> MONITOR[Monitor Logs, Metrics and Errors]
MONITOR --> OK{Healthy?}

OK -- Yes --> RELEASE[Release Complete]
OK -- No --> ROLLBACK[Rollback or Execute Recovery Plan]
```

9.7 Production-readiness requirements

- Environment-specific configuration.
- Secrets stored outside source control.
- HTTPS and appropriate network restrictions.
- Database migrations with a rollback or recovery strategy.
- Automated backups and a tested restore procedure.
- Health and readiness checks.
- Structured logs with correlation or request IDs.
- Metrics and alerts for application failures and resource exhaustion.
- Dependency and container image vulnerability checks where applicable.
- Least-privilege service credentials.
- Documented deployment and rollback procedures.
- Data retention and deletion policies.
- Operational runbook and incident-response guidance.

Module exit criterion: All required tests pass, critical security issues are addressed, recovery procedures are verified, and the production release meets documented operational and performance criteria.

---

10. Cross-Module Architecture Rules

These rules apply to the entire codebase.

10.1 SOLID principles

Single Responsibility Principle

A component should have one clear reason to change.

For example, the research orchestrator coordinates tasks; the evidence validator evaluates claims; the tool adapter communicates with a provider; the report generator builds reports.

Open/Closed Principle

New tools, providers, document parsers and scoring strategies should be introduced through defined interfaces rather than by rewriting core application workflows.

Liskov Substitution Principle

Alternative implementations of an interface must honor its contract. A mock research tool and a live research tool should return compatible result types and failure semantics.

Interface Segregation Principle

Avoid one oversized interface for every integration. A website reader should not be forced to implement people discovery or document parsing methods.

Dependency Inversion Principle

Core business logic should depend on application-owned interfaces, not directly on infrastructure SDKs or provider-specific clients.

10.2 Recommended design patterns

Pattern	Where it belongs	Why
Adapter	External integrations, authentication and document parsing	Isolates vendor-specific interfaces
Strategy	Tool selection, scoring and generation policies	Makes alternatives replaceable
Registry/Factory	Tool discovery and approved implementation selection	Centralizes capability registration
Repository	Research state, evidence, profiles, reports and drafts	Isolates persistence
State	Research lifecycle and approval lifecycle	Makes transitions explicit
Pipeline	Extraction, normalization, validation and report assembly	Separates processing stages
Specification/Policy	Evidence acceptance, authorization and outreach rules	Makes decision criteria explicit

Dependency Injection	Application services	Improves testability and replaceability

Patterns should be used where they simplify the design. Avoid introducing abstractions that do not serve a concrete requirement.

10.3 Dependency direction

```
flowchart TD
    API[API / Presentation]
    APP[Application Services and Use Cases]
    DOMAIN[Domain Models and Business Rules]
    PORTS[Interfaces and Ports]
    INFRA[Infrastructure Adapters]
    DB[(Database)]
    PROVIDERS[External Providers]

    API --> APP
    APP --> DOMAIN
    APP --> PORTS
    INFRA -. implements .-> PORTS
    INFRA --> DB
    INFRA --> PROVIDERS
```

Core business rules should not import provider SDKs or depend on API-framework request objects. Infrastructure implementations should satisfy application-owned contracts.

10.4 Persistence and data lineage

The application should preserve relationships between:

- User and workspace.
- Workspace and seller profile.
- Research job and target company.
- Research job and tool executions.
- Claims and supporting evidence.
- Research version and report version.
- Seller profile version and report version.
- Report and decision-maker records.
- Decision-maker and personalized draft.
- Draft version and approval decision.

Reports and drafts should record the versions of their inputs. This makes it possible to understand why a recommendation was produced and determine when it should be regenerated.

---

11. Shared Data and API Design

11.1 General data design principles

- Use stable identifiers for persistent entities.
- Define ownership and workspace relationships explicitly.
- Apply database constraints to enforce data integrity.
- Use transactions for related writes that must succeed together.
- Design indexes based on actual access patterns.
- Version schemas and scoring policies when their meaning changes.
- Store raw external payloads only when necessary, lawful and safe.
- Keep secrets and unnecessary personal data out of general business records.
- Define retention and deletion rules for research, documents and contact information.
- Use migrations rather than manual production schema changes.

11.2 API design principles

- Use consistent resource naming and HTTP semantics.
- Validate requests at the boundary.
- Return consistent error structures.
- Use pagination for potentially large collections.
- Enforce authorization for every protected resource.
- Use idempotency where duplicate submissions could create unwanted work.
- Use background jobs for long-running research and document processing when appropriate.
- Expose job status and failure details without leaking secrets or internal stack traces.
- Document API contracts and maintain automated contract tests.

A research request should not require a browser connection to remain open until all research completes. Where execution is long-running, return a job identifier and provide a way to query status and results.

---

12. Development and Testing Gates

Each module follows the same engineering cycle.

```
flowchart TD
    REQ[Define Requirements and Acceptance Criteria]
    DESIGN[Write Module Technical Design]
    CONTRACT[Define Models, Interfaces and APIs]
    IMPLEMENT[Implement Small Increments]
    UNIT[Run Unit Tests]
```

```
INTEG[Run Integration and Contract Tests]
REVIEW[Code Review and Security Review]
PASS{Module Gate Passed?}
FIX[Fix Defects and Add Regression Tests]
MERGE[Merge and Update Documentation]
NEXT[Proceed to Next Module]

REQ --> DESIGN --> CONTRACT --> IMPLEMENT
IMPLEMENT --> UNIT --> INTEG --> REVIEW --> PASS
PASS -- No --> FIX --> IMPLEMENT
PASS -- Yes --> MERGE --> NEXT
```

A module is not complete merely because its happy path works. Its acceptance criteria must include failure handling, validation, security and automated tests.

Required artifacts for each module

1. Requirements and acceptance criteria.
2. Module architecture and data flow.
3. Data models and persistence decisions.
4. API or interface contracts.
5. Mermaid diagrams where they clarify the design.
6. SOLID principles and relevant patterns.
7. Implementation tasks and sequencing.
8. Unit, integration, contract and security tests as applicable.
9. Operational and error-handling behavior.
10. Known limitations and decisions requiring later review.

---

13. Recommended Development Sequence

The seven agreed modules remain the product structure. Their implementation order may differ slightly where a dependency requires it.

1. **Authentication & User Management:** Establish identity, workspace ownership and access boundaries.
2. **Tools & Integrations:** Define contracts and implement the initial set of reliable research capabilities.
3. **Lead Research & Orchestration:** Build the evidence-backed, gap-driven research workflow.
4. **Seller Company Profile Setup:** Implement structured seller data and document ingestion. The schema can be designed early, while full ingestion can follow once shared infrastructure is ready.

5. **Summarized Report Generation:** Combine seller context and research evidence into an explainable report.
6. **Personalized Draft Generation:** Use report findings and relevant employee records to generate reviewable drafts.
7. **Production Testing & Deployment:** Validate the integrated system, fix defects, establish operational controls and deploy to the VPS.

The development sequence is not a reason to postpone testing. Every module must pass its own test gate before the next phase relies on it.

---

14. Definition of Product Completion

ProspectIQ is ready for its first production release when:

- Users can securely authenticate and access only authorized workspace data.
- Seller profiles can be created, reviewed and maintained.
- The research tool layer has consistent contracts and safe execution controls.
- Lead research is driven by explicit field requirements and evidence gaps.
- Claims preserve provenance and unsupported facts are not presented as verified.
- Research completion, partial completion and failure are distinguishable.
- Reports combine seller context and lead evidence with explainable scoring.
- Relevant employee records have traceable sources and appropriate confidence.
- Drafts are personalized, validated, versioned and subject to human approval.
- Automated tests cover core logic, integrations, APIs, security and end-to-end workflows.
- Performance, reliability and resource usage have been tested against documented expectations.
- Production configuration, backups, monitoring, deployment and recovery procedures are documented and validated.

---

15. Decisions to Finalize During Detailed Design

This document establishes the intended architecture without prematurely locking in every implementation choice.

The detailed module documents must finalize:

- Authentication provider and workspace permission model.
- Database and document-storage technologies.

- Initial external research and people-data providers.
- Research schema and mandatory fields.
- Evidence quality, freshness and conflict-resolution rules.
- LLM provider, structured-output strategy and evaluation approach.
- Background job execution and recovery strategy.
- Seller–lead scoring formula and policy versioning.
- Supported file types and document extraction approach.
- Contact data policies and supported outreach channels.
- API contracts, deployment topology and measurable production targets.

These decisions should be based on the actual product requirements, provider constraints, expected workload and operational budget.

Conclusion

ProspectIQ will be built as a modular, evidence-driven application with explicit contracts, traceable data, controlled AI behavior and a test gate for every module.

The guiding engineering principle is:

Define the contract, implement the module, verify its behavior, and only then build the next layer on top of it.

This blueprint is the starting point for the detailed module documents. It describes the intended system and development approach; implementation-specific details will be finalized as each module is designed.